

HDOC Day-10 - C Programming

Activities: Algorithm, Pseudocode, Flowchart, Test Cases, C Code, Compilation and Execution

Question 1: Convert Seconds into Days, Hours, Minutes and Seconds

Develop a C program that accepts time in seconds and expresses it in days, hours, minutes, and seconds.

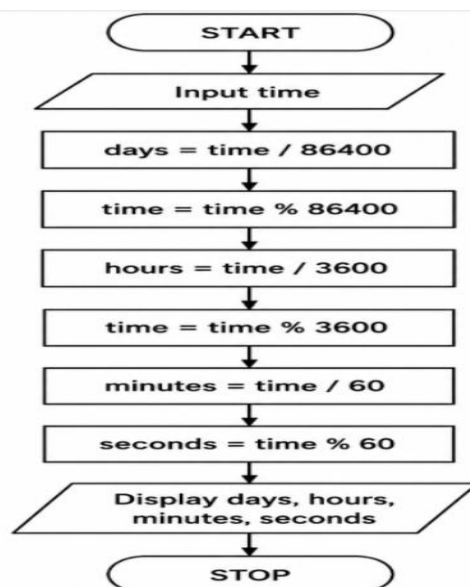
Algorithm

1. Start.
2. Input time in seconds.
3. Calculate $\text{days} = \text{time} / 86400$.
4. Keep the remaining seconds using $\text{time} = \text{time} \% 86400$.
5. Calculate $\text{hours} = \text{time} / 3600$.
6. Keep the remaining seconds using $\text{time} = \text{time} \% 3600$.
7. Calculate $\text{minutes} = \text{time} / 60$.
8. Calculate $\text{seconds} = \text{time} \% 60$.
9. Display days, hours, minutes and seconds.
10. Stop.

Pseudocode

```
START
INPUT time
days = time / 86400
time = time % 86400
hours = time / 3600
time = time % 3600
minutes = time / 60
seconds = time % 60
DISPLAY days, hours, minutes, seconds
STOP
```

Flowchart



Test Case Table

Test Case	Input (seconds)	Days	Hours	Minutes	Seconds
1	60	0	0	1	0
2	3661	0	1	1	1
3	86400	1	0	0	0
4	90061	1	1	1	1

C Program

```
#include <stdio.h>
```

```
int main()
{
    int time, days, hours, minutes, seconds;

    printf("Enter time in seconds: ");
    scanf("%d", &time);

    days = time / 86400;
    time = time % 86400;

    hours = time / 3600;
    time = time % 3600;

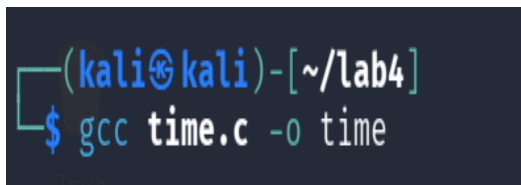
    minutes = time / 60;
    seconds = time % 60;

    printf("Days = %d\n", days);
    printf("Hours = %d\n", hours);
    printf("Minutes = %d\n", minutes);
    printf("Seconds = %d\n", seconds);

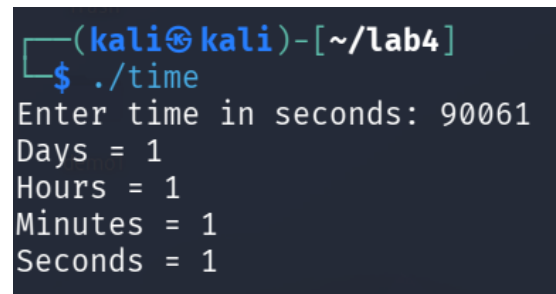
    return 0;
}
```

Compilation and Execution

```
gcc time.c -o time
./time
```



```
(kali@kali)-[~/lab4]
$ gcc time.c -o time
```



```
(kali@kali)-[~/lab4]
$ ./time
Enter time in seconds: 90061
Days = 1
Hours = 1
Minutes = 1
Seconds = 1
```

Result

The program was compiled and executed successfully. It converted the entered seconds into the corresponding days, hours, minutes, and seconds. Testing with the prepared cases confirmed that the obtained results matched the expected output.

Question 2: Check Leap Year

Develop a C program that takes a year as input and determines whether the given year is a leap year.

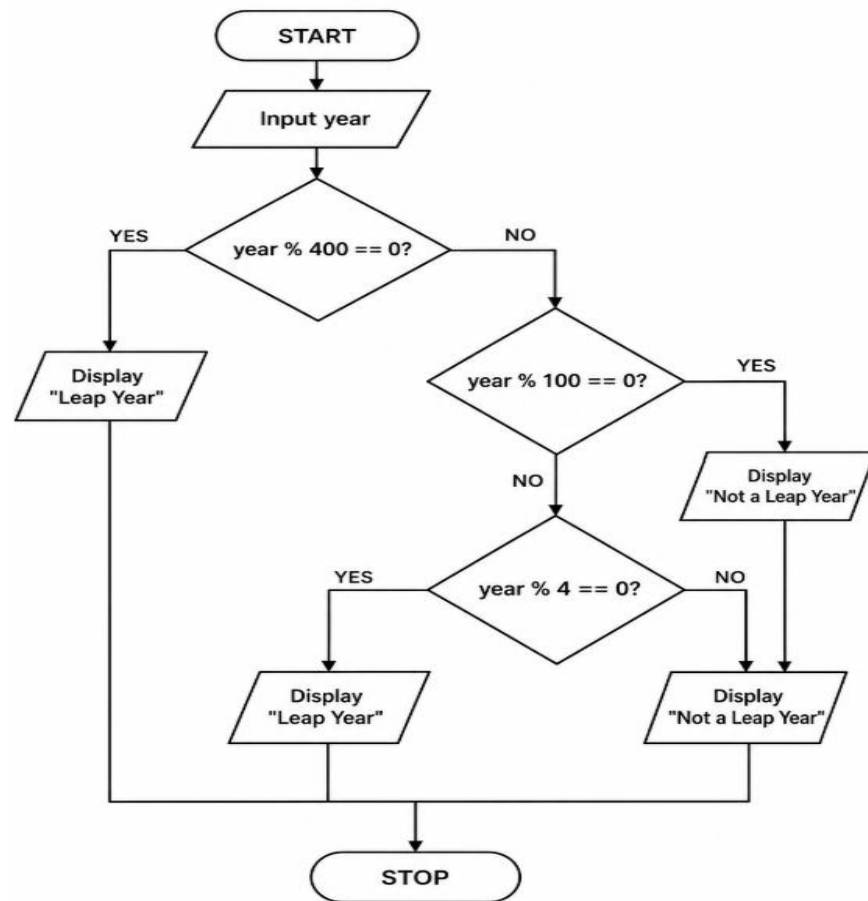
Algorithm

11. Start.
12. Input a year.
13. If the year is divisible by 400, display "Leap Year".
14. Else if the year is divisible by 100, display "Not a Leap Year".
15. Else if the year is divisible by 4, display "Leap Year".
16. Otherwise, display "Not a Leap Year".
17. Stop.

Pseudocode

```
START
INPUT year
IF year % 400 == 0
    DISPLAY "Leap Year"
ELSE IF year % 100 == 0
    DISPLAY "Not a Leap Year"
ELSE IF year % 4 == 0
    DISPLAY "Leap Year"
ELSE
    DISPLAY "Not a Leap Year"
STOP
```

Flowchart



Test Case Table

Test Case	Input Year	Expected Output
1	2024	Leap Year
2	2025	Not a Leap Year
3	1900	Not a Leap Year
4	2000	Leap Year

C Program

```
#include <stdio.h>

int main()
{
    int year;

    printf("Enter year: ");
    scanf("%d", &year);

    if (year % 400 == 0)
        printf("Leap Year");
    else if (year % 100 == 0)
        printf("Not a Leap Year");
    else if (year % 4 == 0)
        printf("Leap Year");
    else
        printf("Not a Leap Year");
}
```

```
    return 0;  
}
```

Compilation and Execution

```
gcc leapyear.c -o leapyear  
./leapyear
```

```
(kali㉿kali)-[~/lab4]  
$ gcc leapyear.c -o leapyear
```

```
(kali㉿kali)-[~/lab4]  
$ ./leapyear
```

Enter year: 2024

Leap Year

```
(kali㉿kali)-[~/lab4]  
$ ./leapyear
```

Enter year: 2025

Not a Leap Year

```
(kali㉿kali)-[~/lab4]  
$ ./leapyear
```

Enter year: 1900

Not a Leap Year

```
(kali㉿kali)-[~/lab4]  
$ ./leapyear
```

Enter year: 2000

Leap Year

Result:

The C program was successfully compiled and executed. It correctly determines whether the entered year is a **leap year or not**. The program was tested using the prepared test cases and produced the expected output.